



Redes Generativas



Dr. Nicolás Gálvez Ramírez
Dr. Patricio Olivares Roncagliolo

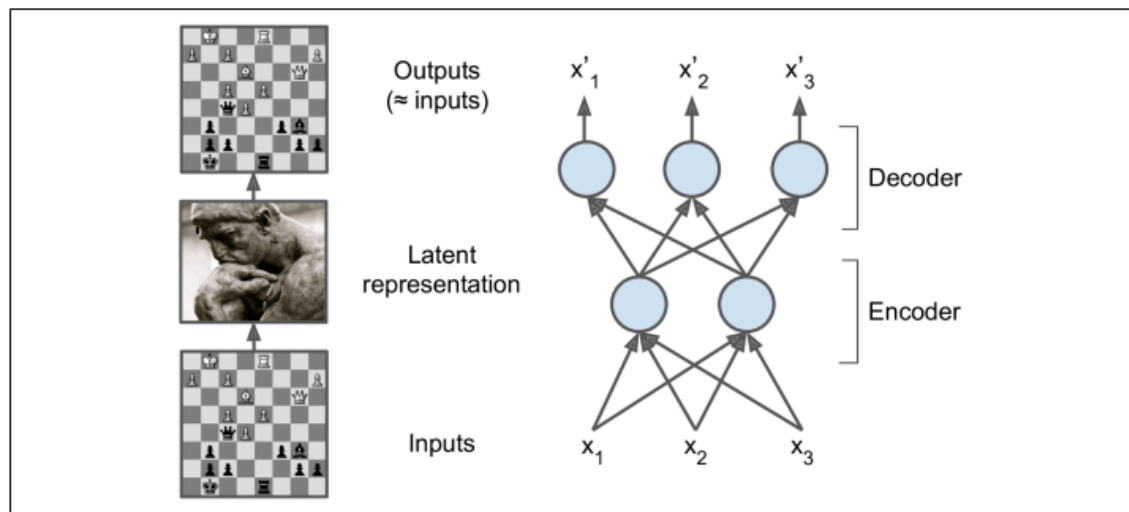
Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

Fuentes:

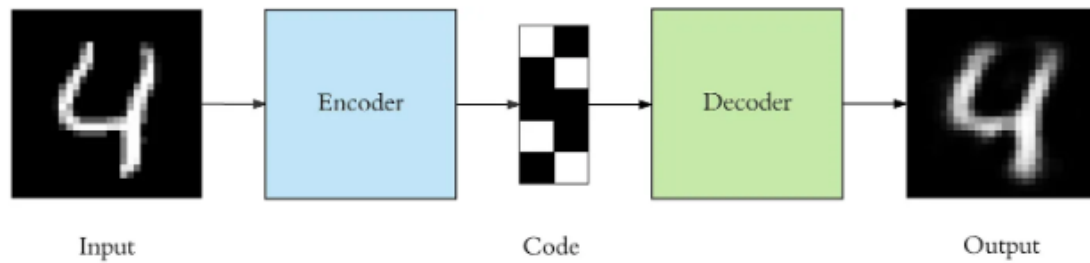
"Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow"

Auto Encoders (AE)

- Redes Neuronales utilizadas para la reducción de dimensionalidad y la extracción de características.
- Consta de dos partes: un **codificador** (coder) y un **decodificador** (decoder).
- El objetivo de un autoencoder es **minimizar la diferencia entre la entrada ya la reconstrucción**



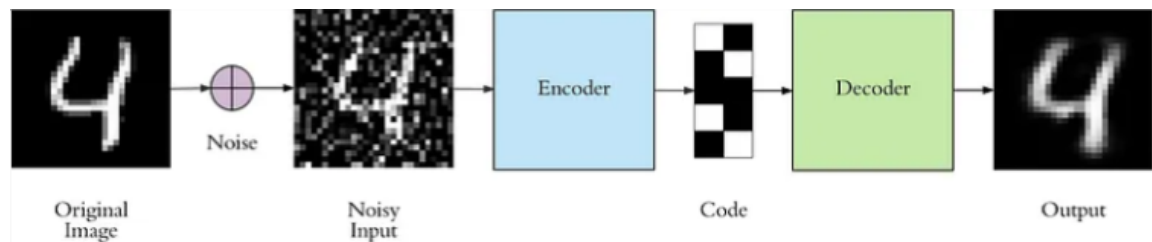
Fuente: "Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow"



Fuente: "<https://towardsdatascience.com/applied-deep-learning-part-3-autoencoders-1c083af4d798>"

Aplicaciones de AutoEncoders

- Reducción de dimensionalidad (compresión)
- Eliminación de ruido en imágenes
- Detección de anomalías



```
In [ ]: import tensorflow as tf
# Ejemplo usando fashion mnist
(X_train_full, y_train_full), (X_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat", "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"] # 10 clases

X_train_full = X_train_full/255.0 # Escalamiento

from sklearn.model_selection import train_test_split
# Utilización de datos de validación y entrenamiento del modelo
X_train, X_val, y_train, y_val = train_test_split(X_train_full, y_train_full, stratify=y_train_full)

stacked_encoder = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dense(100, activation="selu"),
    tf.keras.layers.Dense(30, activation="selu"),
])

stacked_decoder = tf.keras.models.Sequential([
    tf.keras.layers.Dense(100, activation="selu", input_shape=[30]),
    tf.keras.layers.Dense(28 * 28, activation="sigmoid"),
    tf.keras.layers.Reshape([28, 28])
])

stacked_ae = tf.keras.models.Sequential([stacked_encoder, stacked_decoder])
```

```
stacked_ae.compile(loss="binary_crossentropy",  
optimizer=tf.keras.optimizers.SGD(lr=1.5))  
history = stacked_ae.fit(X_train, X_train, epochs=50, validation_data=[X_val
```

2023-11-14 17:27:11.806267: I tensorflow/core/platform/cpu_feature_guard.cc:193] This TensorFlow binary is optimized with oneAPI Deep Neural Network Library (oneDNN) to use the following CPU instructions in performance-critical operations: AVX2 AVX512F AVX512_VNNI FMA
To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.

2023-11-14 17:27:16.356639: I tensorflow/core/util/port.cc:104] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

2023-11-14 17:27:17.170355: W tensorflow/compiler/xla/stream_executor/platform/default/dso_loader.cc:64] Could not load dynamic library 'libnvinfer.so.7'; dlerror: libnvinfer.so.7: cannot open shared object file: No such file or directory; LD_LIBRARY_PATH: /usr/lib/cuda/include:/usr/lib/cuda/lib64:

2023-11-14 17:27:17.170417: W tensorflow/compiler/xla/stream_executor/platform/default/dso_loader.cc:64] Could not load dynamic library 'libnvinfer_plugin.so.7'; dlerror: libnvinfer_plugin.so.7: cannot open shared object file: No such file or directory; LD_LIBRARY_PATH: /usr/lib/cuda/include:/usr/lib/cuda/lib64:

2023-11-14 17:27:17.170421: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Cannot dlopen some TensorRT libraries. If you would like to use Nvidia GPU with TensorRT, please make sure the missing libraries mentioned above are installed properly.

2023-11-14 17:27:18.564014: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero

2023-11-14 17:27:18.564168: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero

2023-11-14 17:27:18.607961: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero

2023-11-14 17:27:18.608101: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero

2023-11-14 17:27:18.608194: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero

2023-11-14 17:27:18.608279: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero

2023-11-14 17:27:18.609304: I tensorflow/core/platform/cpu_feature_guard.cc:193] This TensorFlow binary is optimized with oneAPI Deep Neural Network Library (oneDNN) to use the following CPU instructions in performance-critical operations: AVX2 AVX512F AVX512_VNNI FMA
To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.

2023-11-14 17:27:18.700746: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node

```
zero
2023-11-14 17:27:18.700874: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2023-11-14 17:27:18.700966: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2023-11-14 17:27:18.701047: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2023-11-14 17:27:18.701128: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2023-11-14 17:27:18.701209: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2023-11-14 17:27:19.454152: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2023-11-14 17:27:19.454308: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2023-11-14 17:27:19.456142: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2023-11-14 17:27:19.456242: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2023-11-14 17:27:19.456332: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2023-11-14 17:27:19.456423: I tensorflow/core/common_runtime/gpu/gpu_device.cc:1613] Created device /job:localhost/replica:0/task:0/device:GPU:0 with 2288 MB memory: -> device: 0, name: NVIDIA GeForce RTX 3090, pci bus id: 0000:05:00.0, compute capability: 8.6
2023-11-14 17:27:19.457230: I tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:981] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2023-11-14 17:27:19.457321: I tensorflow/core/common_runtime/gpu/gpu_device.cc:1613] Created device /job:localhost/replica:0/task:0/device:GPU:1 with 4114 MB memory: -> device: 1, name: NVIDIA GeForce RTX 3060 Laptop GPU, pci bus id: 0000:01:00.0, compute capability: 8.6
WARNING:absl:lr is deprecated, please use `learning_rate` instead, or use the legacy optimizer, e.g., tf.keras.optimizers.legacy.SGD.
Epoch 1/50
```

```
2023-11-14 17:27:21.670318: I tensorflow/compiler/xla/stream_executor/cuda/cuda_blas.cc:630] TensorFlow-32 will be used for the matrix multiplication. This will only be logged once.
2023-11-14 17:27:21.964306: I tensorflow/compiler/xla/service/service.cc:173] XLA service 0x7f2a2a42d4d0 initialized for platform CUDA (this does not guarantee that XLA will be used). Devices:
2023-11-14 17:27:21.964323: I tensorflow/compiler/xla/service/service.cc:181] StreamExecutor device (0): NVIDIA GeForce RTX 3090, Compute Capability 8.6
2023-11-14 17:27:21.964326: I tensorflow/compiler/xla/service/service.cc:181] StreamExecutor device (1): NVIDIA GeForce RTX 3060 Laptop GPU, Compute Capability 8.6
2023-11-14 17:27:22.114343: I tensorflow/compiler/jit/xla_compilation_cache.cc:477] Compiled cluster using XLA! This line is logged at most once for the lifetime of the process.
```

```
1500/1500 [=====] - 4s 1ms/step - loss: 0.6638 - va  
l_loss: 0.6118  
Epoch 2/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.5524 - va  
l_loss: 0.5130  
Epoch 3/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.5011 - va  
l_loss: 0.4941  
Epoch 4/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.4881 - va  
l_loss: 0.4825  
Epoch 5/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.4750 - va  
l_loss: 0.4672  
Epoch 6/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.4576 - va  
l_loss: 0.4478  
Epoch 7/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.4374 - va  
l_loss: 0.4273  
Epoch 8/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.4180 - va  
l_loss: 0.4091  
Epoch 9/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.4018 - va  
l_loss: 0.3948  
Epoch 10/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.3899 - va  
l_loss: 0.3852  
Epoch 11/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.3822 - va  
l_loss: 0.3791  
Epoch 12/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.3773 - va  
l_loss: 0.3752  
Epoch 13/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.3741 - va  
l_loss: 0.3724  
Epoch 14/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.3716 - va  
l_loss: 0.3703  
Epoch 15/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.3697 - va  
l_loss: 0.3684  
Epoch 16/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.3679 - va  
l_loss: 0.3668  
Epoch 17/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.3663 - va  
l_loss: 0.3652  
Epoch 18/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.3647 - va  
l_loss: 0.3636  
Epoch 19/50  
1500/1500 [=====] - 2s 1ms/step - loss: 0.3631 - va  
l_loss: 0.3621
```

Epoch 20/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3616 - va
l_loss: 0.3605
Epoch 21/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3600 - va
l_loss: 0.3589
Epoch 22/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3584 - va
l_loss: 0.3573
Epoch 23/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3568 - va
l_loss: 0.3557
Epoch 24/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3552 - va
l_loss: 0.3541
Epoch 25/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3535 - va
l_loss: 0.3525
Epoch 26/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3519 - va
l_loss: 0.3509
Epoch 27/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3504 - va
l_loss: 0.3494
Epoch 28/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3489 - va
l_loss: 0.3479
Epoch 29/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3474 - va
l_loss: 0.3466
Epoch 30/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3461 - va
l_loss: 0.3452
Epoch 31/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3447 - va
l_loss: 0.3440
Epoch 32/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3435 - va
l_loss: 0.3428
Epoch 33/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3423 - va
l_loss: 0.3416
Epoch 34/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3412 - va
l_loss: 0.3405
Epoch 35/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3401 - va
l_loss: 0.3395
Epoch 36/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3390 - va
l_loss: 0.3384
Epoch 37/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3380 - va
l_loss: 0.3374
Epoch 38/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3370 - va


```

l_loss: 0.3364
Epoch 39/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3360 - va
l_loss: 0.3355
Epoch 40/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3351 - va
l_loss: 0.3345
Epoch 41/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3341 - va
l_loss: 0.3336
Epoch 42/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3332 - va
l_loss: 0.3327
Epoch 43/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3323 - va
l_loss: 0.3318
Epoch 44/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3314 - va
l_loss: 0.3310
Epoch 45/50
1500/1500 [=====] - 2s 1ms/step - loss: 0.3306 - va
l_loss: 0.3302
Epoch 46/50
1273/1500 [=====>.....] - ETA: 0s - loss: 0.3302

```

```

In [3]: import matplotlib.pyplot as plt
fig = plt.figure(figsize=(15,15))
prediction = stacked_ae.predict(X_train)

i=19

ax1 = fig.add_subplot(1,2,1)
ax1.imshow(X_train[i], cmap='gist_yarg')
ax1.set_title(class_names[y_train[i]])
ax2 = fig.add_subplot(1,2,2)
ax2.imshow(prediction[i], cmap='gist_yarg')
ax2.set_title(class_names[y_train[i]])

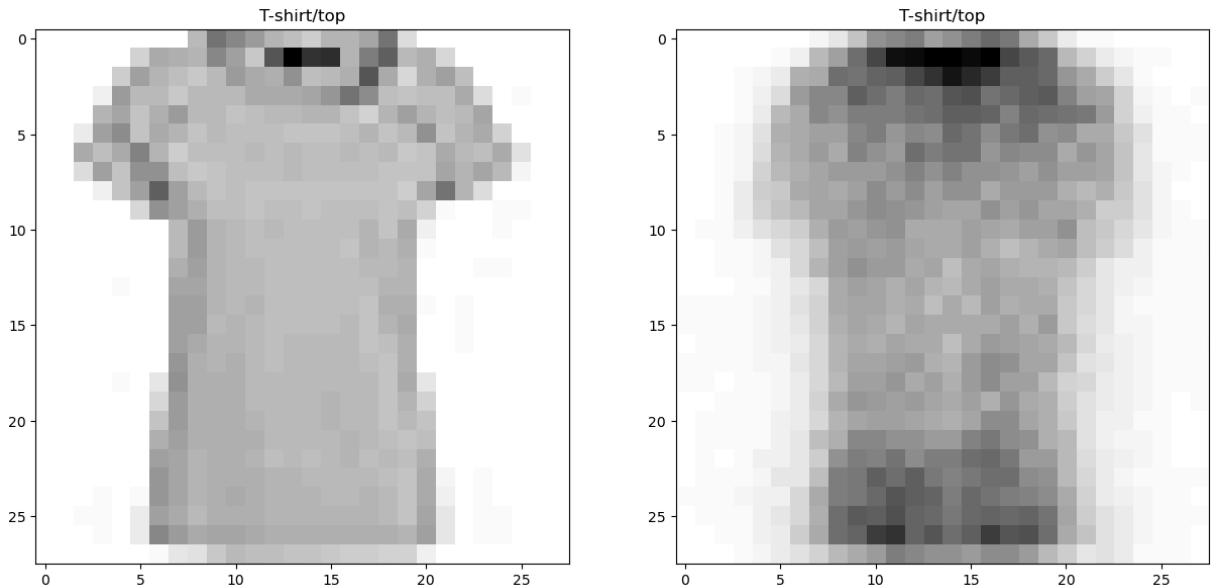
plt.show()

```

```

1500/1500 [=====] - 1s 483us/step

```



Convolutional Auto Encoder

- Autoencoders diseñados para el procesamiento de imágenes.
- Utilizan capas convolucionales.
- Las capas convolucionales permiten capturar patrones en las imágenes de manera más eficiente.

```
In [4]: conv_encoder = tf.keras.models.Sequential([
tf.keras.layers.Reshape([28, 28, 1], input_shape=[28, 28]),
tf.keras.layers.Conv2D(16, kernel_size=3, padding="same", activation="selu"),
tf.keras.layers.MaxPool2D(pool_size=2),
tf.keras.layers.Conv2D(32, kernel_size=3, padding="same", activation="selu"),
tf.keras.layers.MaxPool2D(pool_size=2),
tf.keras.layers.Conv2D(64, kernel_size=3, padding="same", activation="selu"),
tf.keras.layers.MaxPool2D(pool_size=2)
])
conv_decoder = tf.keras.models.Sequential([
tf.keras.layers.Conv2DTranspose(32, kernel_size=3, strides=2, padding="valid",
activation="selu"),
tf.keras.layers.Conv2DTranspose(16, kernel_size=3, strides=2, padding="same",
activation="selu"),
tf.keras.layers.Conv2DTranspose(1, kernel_size=3, strides=2, padding="same",
activation="sigmoid"),
tf.keras.layers.Reshape([28, 28])
])

conv_ae = tf.keras.models.Sequential([conv_encoder, conv_decoder])

conv_ae.compile(loss="binary_crossentropy",
optimizer=tf.keras.optimizers.SGD(lr=1.5))
history = conv_ae.fit(X_train, X_train, epochs=50, validation_data=[X_val, X_val])
```

WARNING:absl:`lr` is deprecated, please use `learning\_rate` instead, or use the legacy optimizer, e.g.,`tf.keras.optimizers.legacy.SGD`.

Epoch 1/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.5851 - va
l_loss: 0.4911
Epoch 2/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.4085 - va
l_loss: 0.3739
Epoch 3/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.3654 - va
l_loss: 0.3550
Epoch 4/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.3505 - va
l_loss: 0.3429
Epoch 5/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.3400 - va
l_loss: 0.3341
Epoch 6/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.3324 - va
l_loss: 0.3274
Epoch 7/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.3264 - va
l_loss: 0.3220
Epoch 8/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.3214 - va
l_loss: 0.3176
Epoch 9/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.3173 - va
l_loss: 0.3139
Epoch 10/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.3139 - va
l_loss: 0.3107
Epoch 11/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.3110 - va
l_loss: 0.3082
Epoch 12/50
1500/1500 [=====] - 5s 4ms/step - loss: 0.3086 - va
l_loss: 0.3059
Epoch 13/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.3065 - va
l_loss: 0.3040
Epoch 14/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.3047 - va
l_loss: 0.3023
Epoch 15/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.3031 - va
l_loss: 0.3009
Epoch 16/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.3017 - va
l_loss: 0.2996
Epoch 17/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.3005 - va
l_loss: 0.2984
Epoch 18/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2993 - va
l_loss: 0.2972
Epoch 19/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2982 - va

```
l_loss: 0.2962
Epoch 20/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2973 - va
l_loss: 0.2953
Epoch 21/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2964 - va
l_loss: 0.2944
Epoch 22/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2955 - va
l_loss: 0.2936
Epoch 23/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2947 - va
l_loss: 0.2928
Epoch 24/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2940 - va
l_loss: 0.2921
Epoch 25/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2933 - va
l_loss: 0.2915
Epoch 26/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2926 - va
l_loss: 0.2908
Epoch 27/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2920 - va
l_loss: 0.2902
Epoch 28/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2914 - va
l_loss: 0.2896
Epoch 29/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2908 - va
l_loss: 0.2891
Epoch 30/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2903 - va
l_loss: 0.2886
Epoch 31/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2898 - va
l_loss: 0.2881
Epoch 32/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2893 - va
l_loss: 0.2876
Epoch 33/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2889 - va
l_loss: 0.2872
Epoch 34/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2884 - va
l_loss: 0.2867
Epoch 35/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2880 - va
l_loss: 0.2863
Epoch 36/50
1500/1500 [=====] - 5s 4ms/step - loss: 0.2876 - va
l_loss: 0.2859
Epoch 37/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2872 - va
l_loss: 0.2855
Epoch 38/50
```

```

1500/1500 [=====] - 5s 3ms/step - loss: 0.2868 - va
l_loss: 0.2851
Epoch 39/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2864 - va
l_loss: 0.2848
Epoch 40/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2861 - va
l_loss: 0.2845
Epoch 41/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2857 - va
l_loss: 0.2841
Epoch 42/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2854 - va
l_loss: 0.2838
Epoch 43/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2851 - va
l_loss: 0.2835
Epoch 44/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2848 - va
l_loss: 0.2832
Epoch 45/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2845 - va
l_loss: 0.2829
Epoch 46/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2842 - va
l_loss: 0.2826
Epoch 47/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2839 - va
l_loss: 0.2823
Epoch 48/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2836 - va
l_loss: 0.2821
Epoch 49/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2834 - va
l_loss: 0.2818
Epoch 50/50
1500/1500 [=====] - 5s 3ms/step - loss: 0.2831 - va
l_loss: 0.2815

```

```

In [6]: import matplotlib.pyplot as plt
fig = plt.figure(figsize=(15,15))
prediction = conv_ae.predict(X_train)

i=7

ax1 = fig.add_subplot(1,2,1)
ax1.imshow(X_train[i], cmap='gist_yarg')
ax1.set_title(class_names[y_train[i]])
ax2 = fig.add_subplot(1,2,2)
ax2.imshow(prediction[i], cmap='gist_yarg')
ax2.set_title(class_names[y_train[i]])

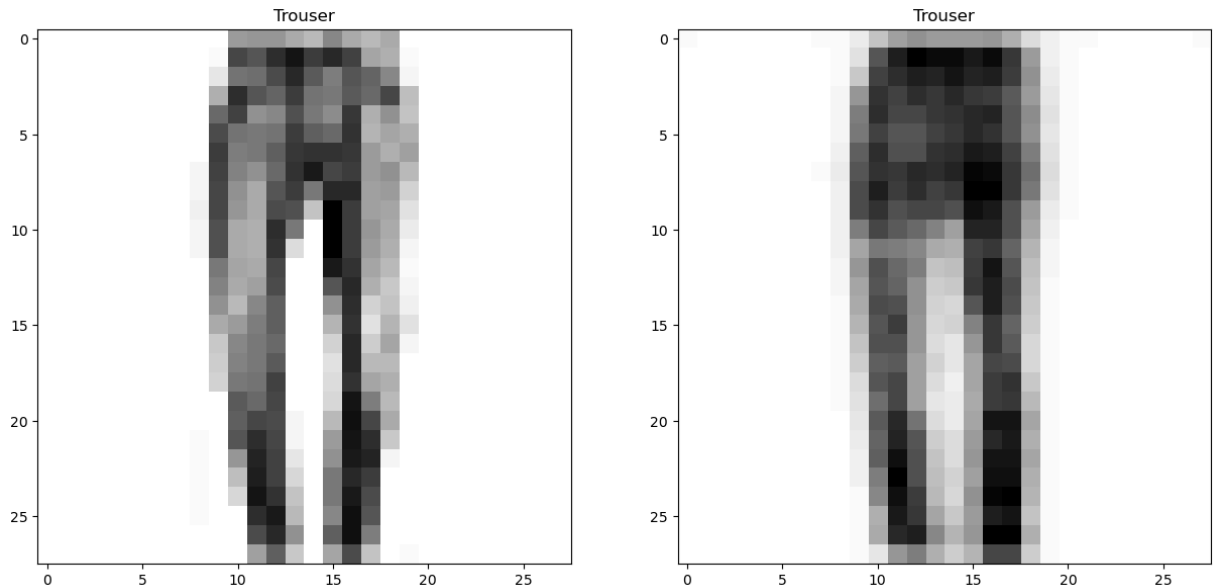
plt.show()

```

```

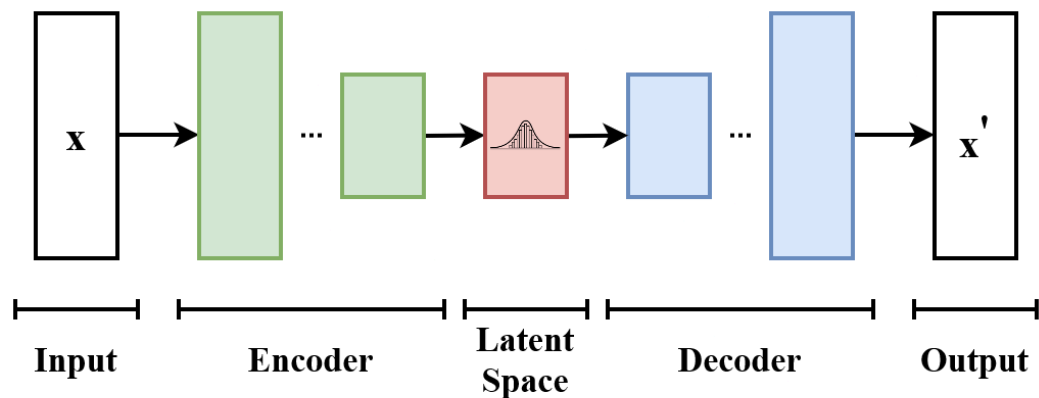
1500/1500 [=====] - 2s 2ms/step

```

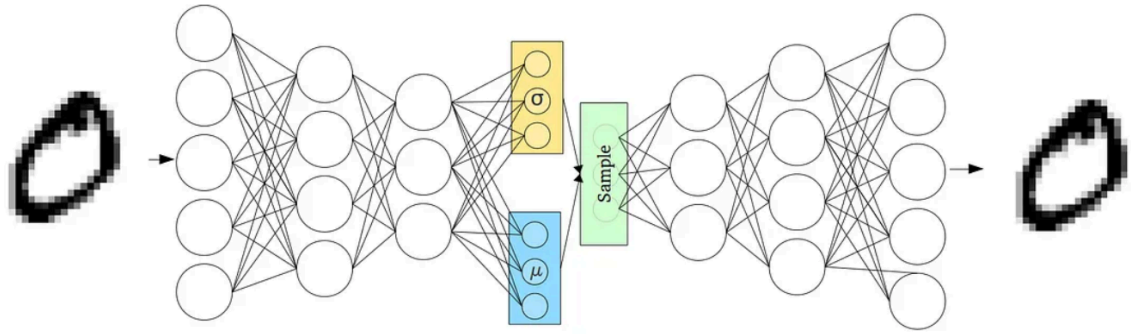


Variational Autoencoders (VAE)

- Un Variational Autoencoder (VAE) es una variante de los autoencoders que combina técnicas de reducción de dimensionalidad y generación de datos con modelos generativos probabilísticos
- Un autoencoder tradicional tiene:
 - Un codificador que reduce la dimensionalidad de los datos de entrada a una representación latente
 - Un decodificador que reconstruye los datos originales a partir de la representación latente.
- En un VAE, también tenemos un codificador y un decodificador, pero se introduce un componente probabilístico en el proceso.



Fuente: "https://en.wikipedia.org/wiki/Variational_autoencoder"



Fuente: "<https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf>"

```
In [7]: import tensorflow as tf
class Sampling(tf.keras.layers.Layer):
    def call(self, inputs):
        mean, log_var = inputs
        return tf.keras.backend.random_normal(tf.shape(log_var)) * tf.keras.

codings_size = 10
inputs = tf.keras.layers.Input(shape=[28, 28])
z = tf.keras.layers.Flatten()(inputs)
z = tf.keras.layers.Dense(150, activation="selu")(z)
z = tf.keras.layers.Dense(100, activation="selu")(z)
codings_mean = tf.keras.layers.Dense(codings_size)(z) # μ
codings_log_var = tf.keras.layers.Dense(codings_size)(z) # γ
codings = Sampling()([codings_mean, codings_log_var])
variational_encoder = tf.keras.Model(
    inputs=[inputs], outputs=[codings_mean, codings_log_var, codings])

decoder_inputs = tf.keras.layers.Input(shape=[codings_size])
x = tf.keras.layers.Dense(100, activation="selu")(decoder_inputs)
x = tf.keras.layers.Dense(150, activation="selu")(x)
x = tf.keras.layers.Dense(28 * 28, activation="sigmoid")(x)
outputs = tf.keras.layers.Reshape([28, 28])(x)
variational_decoder = tf.keras.Model(inputs=[decoder_inputs], outputs=[outputs])
```

```
In [8]: _, _, codings = variational_encoder(inputs)
reconstructions = variational_decoder(codings)
variational_ae = tf.keras.Model(inputs=[inputs], outputs=[reconstructions])

latent_loss = -0.5 * tf.keras.backend.sum(
    1 + codings_log_var - tf.keras.backend.exp(codings_log_var) - tf.keras.backend.square(
        codings_mean), axis=-1)
variational_ae.add_loss(tf.keras.backend.mean(latent_loss) / 784.)
variational_ae.compile(loss="binary_crossentropy", optimizer="rmsprop")
```

```
In [9]: history = variational_ae.fit(X_train, X_train, epochs=50, batch_size=128,
    validation_data=[X_val, X_val])
```

Epoch 1/50
375/375 [=====] - 2s 4ms/step - loss: 0.4208 - val_
loss: 0.3696
Epoch 2/50
375/375 [=====] - 1s 3ms/step - loss: 0.3586 - val_
loss: 0.3469
Epoch 3/50
375/375 [=====] - 1s 3ms/step - loss: 0.3467 - val_
loss: 0.3470
Epoch 4/50
375/375 [=====] - 1s 3ms/step - loss: 0.3398 - val_
loss: 0.3388
Epoch 5/50
375/375 [=====] - 1s 3ms/step - loss: 0.3354 - val_
loss: 0.3296
Epoch 6/50
375/375 [=====] - 1s 3ms/step - loss: 0.3320 - val_
loss: 0.3314
Epoch 7/50
375/375 [=====] - 1s 3ms/step - loss: 0.3295 - val_
loss: 0.3270
Epoch 8/50
375/375 [=====] - 1s 3ms/step - loss: 0.3277 - val_
loss: 0.3235
Epoch 9/50
375/375 [=====] - 1s 3ms/step - loss: 0.3262 - val_
loss: 0.3240
Epoch 10/50
375/375 [=====] - 1s 3ms/step - loss: 0.3248 - val_
loss: 0.3217
Epoch 11/50
375/375 [=====] - 1s 3ms/step - loss: 0.3237 - val_
loss: 0.3201
Epoch 12/50
375/375 [=====] - 1s 4ms/step - loss: 0.3228 - val_
loss: 0.3218
Epoch 13/50
375/375 [=====] - 1s 3ms/step - loss: 0.3220 - val_
loss: 0.3209
Epoch 14/50
375/375 [=====] - 1s 4ms/step - loss: 0.3212 - val_
loss: 0.3198
Epoch 15/50
375/375 [=====] - 1s 3ms/step - loss: 0.3206 - val_
loss: 0.3170
Epoch 16/50
375/375 [=====] - 1s 3ms/step - loss: 0.3199 - val_
loss: 0.3179
Epoch 17/50
375/375 [=====] - 1s 3ms/step - loss: 0.3194 - val_
loss: 0.3179
Epoch 18/50
375/375 [=====] - 1s 3ms/step - loss: 0.3189 - val_
loss: 0.3164
Epoch 19/50
375/375 [=====] - 1s 3ms/step - loss: 0.3185 - val_


```
loss: 0.3198
Epoch 20/50
375/375 [=====] - 1s 3ms/step - loss: 0.3180 - val_
loss: 0.3171
Epoch 21/50
375/375 [=====] - 1s 3ms/step - loss: 0.3176 - val_
loss: 0.3149
Epoch 22/50
375/375 [=====] - 1s 3ms/step - loss: 0.3172 - val_
loss: 0.3154
Epoch 23/50
375/375 [=====] - 1s 4ms/step - loss: 0.3169 - val_
loss: 0.3146
Epoch 24/50
375/375 [=====] - 1s 3ms/step - loss: 0.3166 - val_
loss: 0.3140
Epoch 25/50
375/375 [=====] - 1s 3ms/step - loss: 0.3163 - val_
loss: 0.3147
Epoch 26/50
375/375 [=====] - 1s 3ms/step - loss: 0.3160 - val_
loss: 0.3148
Epoch 27/50
375/375 [=====] - 1s 3ms/step - loss: 0.3157 - val_
loss: 0.3157
Epoch 28/50
375/375 [=====] - 1s 4ms/step - loss: 0.3155 - val_
loss: 0.3138
Epoch 29/50
375/375 [=====] - 1s 4ms/step - loss: 0.3152 - val_
loss: 0.3128
Epoch 30/50
375/375 [=====] - 1s 3ms/step - loss: 0.3150 - val_
loss: 0.3156
Epoch 31/50
375/375 [=====] - 1s 3ms/step - loss: 0.3148 - val_
loss: 0.3131
Epoch 32/50
375/375 [=====] - 1s 4ms/step - loss: 0.3146 - val_
loss: 0.3139
Epoch 33/50
375/375 [=====] - 1s 4ms/step - loss: 0.3144 - val_
loss: 0.3124
Epoch 34/50
375/375 [=====] - 1s 4ms/step - loss: 0.3142 - val_
loss: 0.3123
Epoch 35/50
375/375 [=====] - 1s 4ms/step - loss: 0.3140 - val_
loss: 0.3120
Epoch 36/50
375/375 [=====] - 1s 4ms/step - loss: 0.3138 - val_
loss: 0.3151
Epoch 37/50
375/375 [=====] - 1s 3ms/step - loss: 0.3136 - val_
loss: 0.3127
Epoch 38/50
```

```

375/375 [=====] - 1s 3ms/step - loss: 0.3135 - val_
loss: 0.3117
Epoch 39/50
375/375 [=====] - 1s 4ms/step - loss: 0.3133 - val_
loss: 0.3113
Epoch 40/50
375/375 [=====] - 1s 3ms/step - loss: 0.3132 - val_
loss: 0.3111
Epoch 41/50
375/375 [=====] - 1s 3ms/step - loss: 0.3130 - val_
loss: 0.3103
Epoch 42/50
375/375 [=====] - 1s 4ms/step - loss: 0.3129 - val_
loss: 0.3113
Epoch 43/50
375/375 [=====] - 1s 4ms/step - loss: 0.3127 - val_
loss: 0.3109
Epoch 44/50
375/375 [=====] - 1s 4ms/step - loss: 0.3126 - val_
loss: 0.3117
Epoch 45/50
375/375 [=====] - 1s 4ms/step - loss: 0.3125 - val_
loss: 0.3119
Epoch 46/50
375/375 [=====] - 1s 3ms/step - loss: 0.3123 - val_
loss: 0.3099
Epoch 47/50
375/375 [=====] - 1s 4ms/step - loss: 0.3122 - val_
loss: 0.3114
Epoch 48/50
375/375 [=====] - 1s 3ms/step - loss: 0.3121 - val_
loss: 0.3104
Epoch 49/50
375/375 [=====] - 1s 4ms/step - loss: 0.3120 - val_
loss: 0.3103
Epoch 50/50
375/375 [=====] - 1s 4ms/step - loss: 0.3119 - val_
loss: 0.3097

```

```

In [13]: codings = tf.random.normal(shape=[12, codings_size])
         images = variational_decoder(codings).numpy()

         import matplotlib.pyplot as plt
         fig = plt.figure(figsize=(3,3))
         prediction = conv_ae.predict(X_train)

         i=8

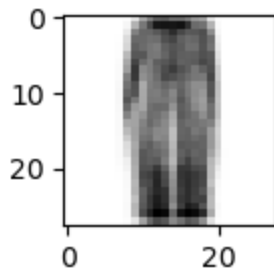
         ax1 = fig.add_subplot(1,2,1)
         ax1.imshow(images[i], cmap='gist_yarg')
         plt.show()

```

```

1500/1500 [=====] - 2s 1ms/step

```

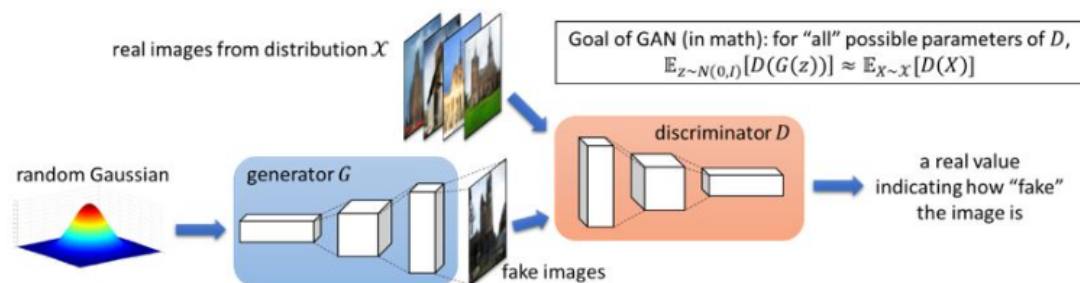


En los VAE, el codificador crea instancias a partir del muestreo de las variables latentes, lo que habilita a estos modelos para generar datos distintos de los que se utilizaron en su entrenamiento inicial.

Generative Adversarial Networks (GAN)

Las Redes Generativas Adversarias (GAN por sus siglas en inglés), es una arquitectura de red neuronal que se utiliza para generar datos. Consta de dos componentes:

- El Generador: El cual crea nuevos datos a partir de un ruido aleatorio o un vector de entrada
- El Discriminador: Este evalúa si un dato es real (proveniente de un conjunto de entrenamiento real) o falso (generado por el generador)



Fuente: "<https://www.microsoft.com/en-us/research/blog/how-can-generative-adversarial-networks-learn-real-life-distributions-easily/>"

Entrenamiento de una red GAN

El entrenamiento consta de dos etapas:

- **Entrenamiento del discriminador:** En la primera fase, alimentamos al generador con ruido gaussiano para producir imágenes falsas. Completamos este lote concatenando un número igual de imágenes reales. Las etiquetas se establecen en 0 para imágenes falsas y 1 para imágenes reales. Luego, entrenamos al discriminador en este lote.

- **Entrenamiento del generador:** En la segunda fase, suministramos ruido gaussiano al modelo GAN. El generador producirá inicialmente imágenes falsas y el discriminador intentará determinar si estas imágenes son falsas o reales. Queremos que el discriminador crea que las imágenes falsas son reales, por lo que las etiquetas se establecen en 1.

```
In [9]: import tensorflow as tf
codings_size = 30
generator = tf.keras.models.Sequential([
    tf.keras.layers.Dense(100, activation="selu", input_shape=[codings_size]),
    tf.keras.layers.Dense(150, activation="selu"),
    tf.keras.layers.Dense(28 * 28, activation="sigmoid"),
    tf.keras.layers.Reshape([28, 28])
])

discriminator = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dense(150, activation="selu"),
    tf.keras.layers.Dense(100, activation="selu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

gan = tf.keras.models.Sequential([generator, discriminator])

discriminator.compile(loss="binary_crossentropy", optimizer="rmsprop")
discriminator.trainable = False
gan.compile(loss="binary_crossentropy", optimizer="rmsprop")

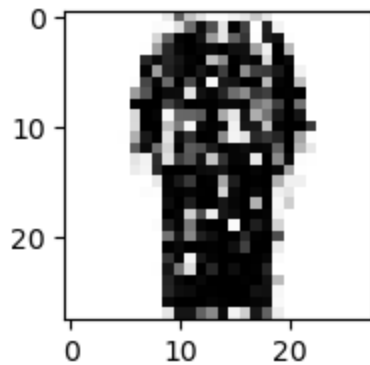
batch_size = 32
dataset = tf.data.Dataset.from_tensor_slices(X_train).shuffle(1000)
dataset = dataset.batch(batch_size, drop_remainder=True).prefetch(1)
```

```
In [10]: from tqdm import tqdm
def train_gan(gan, dataset, batch_size, codings_size, n_epochs=50):
    generator, discriminator = gan.layers
    for epoch in tqdm(range(n_epochs)):
        for X_batch in dataset:
            # phase 1 - training the discriminator
            noise = tf.random.normal(shape=[batch_size, codings_size])
            generated_images = tf.cast(generator(noise), tf.float32)
            X_batch = tf.cast(X_batch, tf.float32)
            X_fake_and_real = tf.concat([generated_images, X_batch], axis=0)
            y1 = tf.constant([[0.]] * batch_size + [[1.]] * batch_size)
            discriminator.trainable = True # Aquí el discriminador se marca como trainable
            discriminator.train_on_batch(X_fake_and_real, y1)
            # phase 2 - training the generator
            noise = tf.random.normal(shape=[batch_size, codings_size])
            y2 = tf.constant([[1.]] * batch_size)
            discriminator.trainable = False
            gan.train_on_batch(noise, y2)
    train_gan(gan, dataset, batch_size, codings_size)
```

```
In [11]: noise = tf.random.normal(shape=[1, codings_size])
image = generator(noise).numpy().reshape((28,28))

import matplotlib.pyplot as plt
fig = plt.figure(figsize=(3,2))

ax1 = fig.add_subplot(1,1,1)
ax1.imshow(image, cmap='gist_yarg')
plt.show()
```



```
In [12]: from IPython.core.display import HTML
HTML("""
<style>
.output_png {
    display: table-cell;
    text-align: center;
    vertical-align: middle;
}
</style>
""")
#codigo extra, para que imagenes de matplotlib
#estén centradas en las diapositivas, ejecutar antes de lanzar los ejemplos.
```

Out[12]: